Organización y Programación de Computadoras

Funciones trigonométricas en Ensamblador x86

José Alberto Márquez Luján, 187917

2026

1. Introducción

Este reporte describe el diseño, implementación y análisis de un programa escrito enteramente en lenguaje ensamblador x86 que calcula las funciones trigonométricas seno y coseno. Los cálculos se hacen con la expansión de series de Taylor y se evalúan en la unidad de aritmética de punto flotante (FPU por sus siglas en inglés: *Floating-Point Unit*). Para conocimientos sobre el lenguaje ensamblador x86, seguimos a Irvine (2015) y las notas de la clase de Organización y programación de computadoras de Ríos (2023).

2. Teoría

Empecemos por describir algunos fundamentos básicos del lenguaje ensamblador x86. Para un entendimiento profundo de los conceptos aquí discutidos, sugerimos fuertemente revisar a detalle el libro de Irvine (2015).

2.1. Lenguaje ensamblador

Cada instrucción que ejecuta un procesador es una secuencia de bytes, una cadena binaria que los humanos difícilmente podemos entender directamente. Escribir programas directamente en lenguaje de máquina —las cadenas binarias— no es una tarea práctica para los humanos, por lo que existen abstracciones que facilitan el trabajo con las computadoras. Una de las primeras abstracciones que se desarrollaron fue el lenguaje ensamblador, que ofrece una correspondencia uno a uno (o casi uno a uno) entre instrucciones mnemónicas (que podemos leer) y sus codificaciones en lenguaje de máquina.

Con esto, los humanos escribimos los mnemónicos, un ensamblador (en este proyecto, UASM) traduce los mnemónicos a código objeto, y un enlazador combina los archivos objeto para producir un ejecutable final. A diferencia de los lenguajes de alto nivel, en el lenguaje ensamblador el programador es responsable de cosas que normalmente damos por hecho: el manejo del *stack*, el paso de parámetros entre procedimientos, la preservación de registros, etcétera.

Hay directivas como `.DATA`, `.CODE`, `PROC` que no producen instrucciones de máquina, pero guían al ensamblador en la organización de la salida. También hay directivas como `.IF`/`.ENDIF` que se traducen automáticamente a saltos condicionales y etiquetas, lo cual facilita considerablemente la legibilidad del código.

Como menciona Irvine (2015), los programadores que usan lenguaje ensamblador necesitan manejar los datos a un nivel físico, por lo que debemos acostumbrarnos a examinar memoria y registros. Veamos ahora cómo se puede almacenar a los datos, empezando por los datos enteros.

2.2. Tipos de datos enteros

La arquitectura x86 soporta enteros de varios tamaños, cada uno con su correspondiente directiva en MASM para reservar e inicializar memoria. Los valores se almacenan en binario (nuevamente, secuencias de unos y ceros), y la unidad básica es el *byte*, que concatena 8 bits; cada bit puede tomar el valor de uno o cero. La tabla en la figura 1 contiene a tres tamaños de enteros comunes.

Directiva	Tamaño	Rango (con signo)	Rango (sin signo)
`BYTE`	8 bits	−128 a 128	0 a 255
`WORD`	16 bits	−32,768 a 32,767	0 a 65,535
`DWORD`	32 bits	−2,147,483,648 a 2,147,483,647	0 a 4,294,967,295

Figura 1: Enteros en MASM.

Los enteros con signo utilizan una representación en complemento a dos, donde el bit más significativo funciona como bit de signo. Pero cabe aclarar que se almacenan igual qeu los enteros sin signo; es responsabilidad del programador saber con qué tipo de dato está trabajando. La distinción importa especialmente al momento de hacer comparaciones condicionales, pero las directivas ayudan mucho con esto.

En este proyecto, los datos enteros aparecen principalmente en dos contextos: el contador de iteraciones en las series de Taylor, almacenado como `DWORD` y los valores en grados, que también son almacenados como `DWORD`. También usamos cadenas de caracteres como sucesiones de `BYTE` que terminan en un byte nulo para los mensajes que se imprimen al usuario.

Para reservar espacio sin inicializar un valor, usamos el operador `?`; por ejemplo, `deg_val DWORD ?`. Para cuando sí queremos asignar un valor, basta con escribirlo literalmente, como en `val180 DWORD 180`.

2.3. Tipos de datos de punto flotante

El estándar IEEE 754 define el formato de los números de punto flotante en binario que utilizan los procesadores x86. La idea es que un número de punto flotante se construye a partir de tres componentes: el signo, el exponente y el significando (también llamado mantisa). Por ejemplo, en el número -1.23154×10^5, el signo es negativo, el significando es 1.23154 y el exponente es 5 (Irvine, 2015).

La arquitectura x86 soporta tres formatos principales, descritos en la tabla de la figura 2.

Precisión	Total de bits	S, E, F	Rango aproximado
Simple (*short real*)	32	1, 8, 23	2^{-126} a 2^{127}
Doble (*long real*)	64	1, 11, 52	2^{-1022} a 2^{1023}
Extendida (*extended real*)	80	1, 16, 63	2^{-16382} a 2^{16383}

Figura 2: Formatos del IEEE para binarios en punto flotante.

Como podemos observar, el signo siempre se codifica en un bit. Si el bit es 1, el número es negativo; si es 0, el número es positivo. Cada uno de los tipos de precisión que describimos en la tabla de la figura 2 tiene un tipo de dato que soporta MASM: `REAL4` corresponde a un real de 4 bytes (32 bits, precisión simple); `REAL8` a uno de precisión doble y `REAL10` a uno de precisión extendida. En este proyecto, usamos `REAL8` para todos los valroes de punto flotante que almacenamos en memoria y toda la aritmética se realiza en precisión extendida de 80 bits sobre el stack del FPU.

El significando de un número de punto flotante consiste en los dígitos a la izquierda y derecha del punto decimal. Para representarlo, usamos el mismo concepto que para convertir números

enteros. Por ejemplo, el decimal 123.154 se representa por la siguiente suma:

$$123.154 = (1 \times 10^2) + (2 \times 10^1) + (3 \times 10^0) + (1 \times 10^{-1}) + (5 \times 10^{-2}) + (4 \times 10^{-3})$$

Todos los dígitos a la izquierda del punto decimal tienen exponentes positivos y los de la derecha tienen exponentes negativos. Los números binarios de punto flotante usan el mismo concepto. Por ejemplo, el valor binario 11.1011 es

$$11.1011 = (1 \times 2^1) + (1 \times 2^0) + (1 \times 2^{-1}) + (0 \times 10^{-2}) + (1 \times 10^{-3}) + (1 \times 2^{-4}).$$

Es imposible representar a todos los números reales con un número finito de bits. Por ejemplo, si contamos con 5 bits para el significando, no tenemos manera de representar los valores entre 1.1111 y 10.000 en binario (Irvine, 2015). Para mitigar un poco esto, los números binarios de punto flotante se almacenan en una forma normalizada, que permite maximizar la precisión del significando. La normalización consiste en recorrer el punto binario hasta que un solo 1 aparezca a la izquierda del punto; como ese dígito siempre es un 1, el estándar no lo almacena en memoria. Gracias a esto, un campo de 23 bits de fracción ofrece 24 bits de precisión efectiva en el significando. La tabla de la figura 3, tomada de Irvine (2015), muestra unos ejemplos.

Número	Normalización
1110.1	1.1101×2^3
.000101	1.01×2^{-4}
1010001.	1.010001×2^6

Figura 3: Valores normalizados.

Para representar exponentes negativos sin necesidad de un bit de signo adicional, se utiliza un sesgo que se suma al exponente real. Para precisión simple, el sesgo es de 127, de manera que el exponente sesgado vive en el rango de 1 a 254. Los valores 0 y 255 están reservados para codificaciones especiales, de manera que el rango de exponentes verdaderos va de -126 a $+127$.

Una vez que tenemos el bit de signo, el exponente, y el significando, es fácil generar un binario en punto flotante como lo define el estándar IEEE. por ejemplo, el binario +1101.101 es positivo, entonces tiene bit de signo 0. Para normalizarlo, tendríamos que mover el punto decimal tres lugares a la izquierda, $(+1.101101 \times 2^3)$, por lo que el exponente es 3 y el exponente sesgado es $127 + 3 = 130$, o 10000010 en binario. Si quitamos el 1 de la izquierda del punto decimal, el significando simplemente queda como 101101. Por lo tanto, la representación en binario de punto flotante con precisión simple de +1101.101 es

$$0 \quad 10000010 \quad 10110100000000000000000.$$

2.4. Arquitectura del FPU y registros

El FPU es un coprocesador integrado en cada CPU x86 desde el Intel486 (Irvine, 2015). Su arquitectura difiere fundamentalmente de la de los registros de propósito general: en lugar de un banco de registros direccionables individualmente, el FPU está optimizado para evaluar expresiones en notación postfija.

El FPU contiene ocho registros de 80 bits, denotados $R0$ a $R7$, organizados como una pila circular. Un campo llamado TOP de 3 bits indica cuál de los registros es el *top* actual del

stack, al que nos referimos como ST(0). Los demás registros se direccionan relativamente al *top*: ST(1), ST(2), etc.

La dinámica del *stack* es la siguiente:

- *Push*: TOP se decrementa en 1 y el valor se almacena en el nuevo ST(0).
- *Pop*: se copia lo almacenado en ST(0) a un operando y luego se incrementa TOP en 1.
- Naturaleza circular: si TOP es 0 antes de un *push*, TOP se va a 7; si TOP es 7 antes de un *pop*, se va a 0. Hacer *push* a un stack lleno dispara una excepción.

Con esto, concluimos la breve descripción teórica del lenguaje ensamblador. Revisemos ahora las matemáticas que sostienen el código desarrollado en este proyecto.

3. Conceptos matemáticos

Para implementar las funciones de seno y coseno, en lenguaje ensamblador, nos tenemos que concentrar en operaciones básicas. Es por esto que nos basaremos en las series de Taylor para hacer los cálculos. En esta sección, veremos las matemáticas que sostienen este proyecto.

3.1. Series de Taylor para seno y coseno

Según Thomas (2010), si f es una función con derivadas de todos los órdenes en algún intervalo que contenga a a como punto interior, entonces la **serie de Taylor generada por f en $x = a$** es

$$\sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!}(x-a)^k = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \cdots, \tag{1}$$

y la **serie de Maclaurin generada por f** es

$$\sum_{k=0}^{\infty} \frac{f^{(k)}(0)}{k!}x^k = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \cdots, \tag{2}$$

que es la serie de Taylor generada por f en $x = 0$.

Thomas (2010) menciona que, con frecuencia, la serie de Maclaurin generada por f sólo se llama serie de Taylor de f. Nosotros seguiremos esa convención en este reporte.

No es difícil ver que la serie de Taylor para $\sin(x)$ en $x = 0$ es

$$\sum_{k=0}^{\infty} \frac{(-1)^k x^{2k+1}}{(2k+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots, \tag{3}$$

mientras que la serie de Taylor para $\cos(x)$ en $x = 0$ es

$$\sum_{k=0}^{\infty} \frac{(-1)^k x^{2k}}{(2k)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \cdots. \tag{4}$$

3.2. Residuos y convergencia

El teorema 24 de Thomas (2010) nos dice que, si existe una constante positiva M tal que $|f^{(n+1)}(t)| \leq M$ para toda t entre x y a, inclusive, entonces el término residual $R_n(x)$ del teorema de Taylor satisface la desigualdad

$$|R_n(x)| \leq M\frac{|x-a|^{n+1}}{(n+1)!}. \tag{5}$$

y, si esta desigualdad se cumple para toda n y f satisface todas las demás condiciones del teorema de Taylor, entonces la serie converge a $f(x)$.

Como todas las derivadas de $\sin(x)$ tienen valores absolutos menores o iguales que 1, entonces podemos aplicar (5) con $M = 1$ para obtener

$$|R_{2k+1}(x)| \leq \frac{|x|^{2k+2}}{(2k+2)!}. \tag{6}$$

Como el lado derecho de (6) tiende a cero cuando $k \to \infty$, entonces $|R_{2k+1}(x)| \to 0$ y la serie de Taylor para $\sin(x)$ converge a $\sin(x)$ para toda x. Por lo tanto,

$$\sin(x) = \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k+1}}{(2k+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots. \tag{7}$$

Análogamente, podemos aplicar (5) con $M = 1$ para $\cos(x)$:

$$|R_{2k}(x)| \leq \frac{|x|^{2k+1}}{(2k+1)!}. \tag{8}$$

De la misma manera que antes, $R_{2k} \to 0$ cuando $k \to \infty$. Por lo tanto, la serie converge a $\cos(x)$ para todo valor de x, lo que quiere decir que

$$\cos(x) = \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k}}{(2k)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \cdots. \tag{9}$$

Notemos que, aunque ambas series convergen para toda x —lo que quiere decir que eventualmente producirán un resultado cercano a la función, sin importar la entrada—, la convergencia depende de manera importante de $|x|$. Si $|x| < 1$, cada término sucesivo es notablemente más pequeño que el anterior, por lo que las series convergen rápidamente. Aunque no es necesario que $|x| < 1$, sí conviene que $|x|$ sea pequeño. Por ejemplo, para $\sin(x)$ con $k = 10$ y $|x| \leq \frac{\pi}{2}$, por (6) tenemos que

$$|R_{2(10)+1}(x)| \leq \frac{|x|^{22}}{(22)!} \leq \frac{|\pi/2|^{22}}{(22)!} \approx 1.836 \times 10^{-17}, \tag{10}$$

lo cual es una precisión bastante razonable. Sin embargo, si realizamos el mismo cálculo con $x = 2\pi$, o bien, 360°, tenemos que

$$|R_{2(10)+1}(2\pi)| \leq \frac{|2\pi|^{22}}{(22)!} \approx 3.23 \times 10^{-4}, \tag{11}$$

lo que implica una pérdida de precisión bastante considerable. Este hecho motivó, como veremos más adelante, el ajuste de rango al momento de convertir grados a radianes.

4. Diseño del programa

Ya que revisamos la teoría matemática necesaria para desarrollar este proyecto, veremos ahora el diseño y la implementación del código.

4.1. Estructura modular

El código está dividido en 11 archivos con terminación `.asm`. Cada archivo contiene uno o dos procedimientos relacionados entre sí. En la tabla 4, presentamos una tabla con los procedimientos que tiene cada archivo y una breve descripción.

Módulo	Procedimiento(s)	Descripción
`main.asm`	`main`, `ImprimeMenu`	Punto de entrada, bucle de menú, texto compartido.
`pide-radianes.asm`	`PideRadianes`	Se encarga de pedir los radianes al usuario y leer la entrada.
`pide-grados.asm`	`PideGrados`	Se encarga de pedir los grados al usuario y leer la entrada.
`tabla-grados.asm`	`TablaGrados`	Se encarga de llamar a las funciones para calcular una tabla con valores de 0° a 360° en pasos de 10°.
`eval-sin-cos.asm`	`EvalSinCos`	Dados los radianes, se encarga de llamar a los procedimientos para calcular seno, coseno e identidad trigonométrica.
`eval-grados.asm`	`EvalGrados`	Dados los grados, se encarga de llamar a los procedimientos para convertir a radianes y luego evaluar.
`convierte-rango.asm`	`ConvierteRangoSin`, `ConvierteRangoCos`	Convierte grados a radianes y aplica una reducción de rango.
`imprime-grados.asm`	`ImprimeGrados`	Se encarga del formato para imprimir los resultados cuando se manejan grados.
`sine-n.asm`	`SinFloat`	Evalúa n términos de la serie de Taylor para $\sin(x)$, con x en radianes.
`cosine-n.asm`	`CosFloat`	Evalúa n términos de la serie de Taylor para $\cos(x)$, con x en radianes.
`square-float.asm`	`SquareFloat`	Se encarga de elevar al cuadrado el valor en ST(0).

Figura 4: Descripción de módulos.

4.2. Diagrama de dependencias

El diagrama 5 muestra la relación entre los diferentes procedimientos del programa.

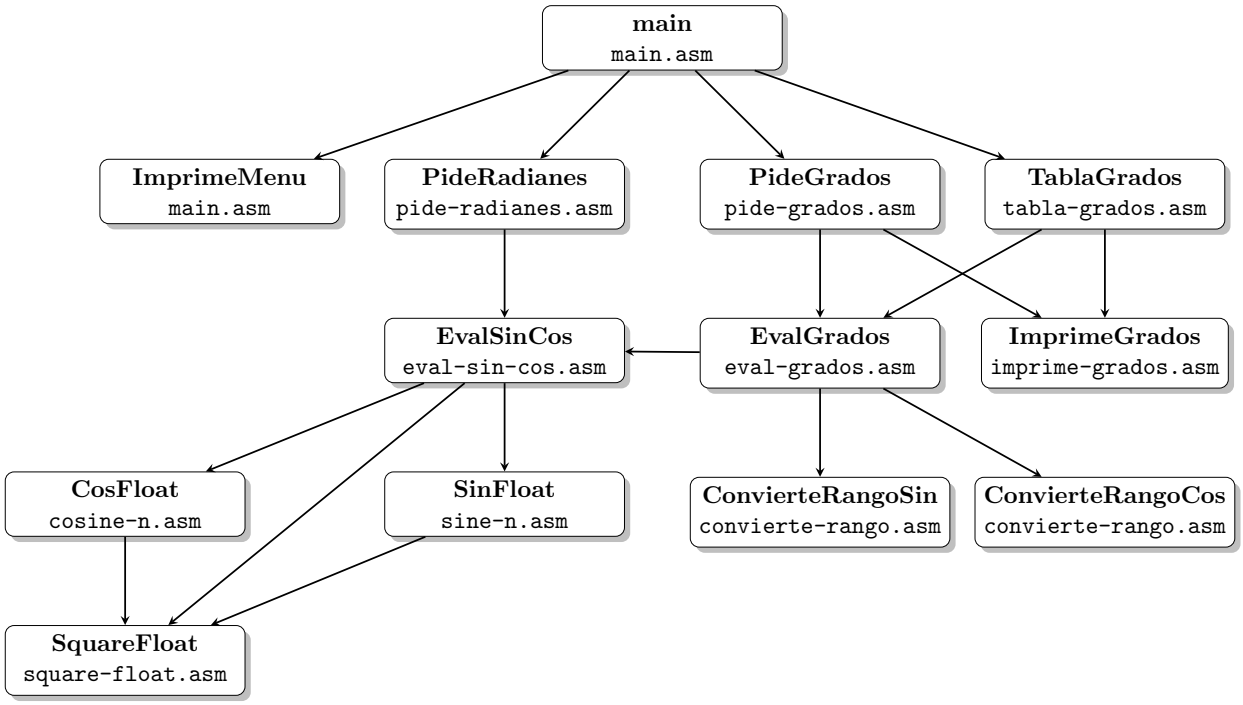


Figura 5: Diagrama de dependencias.

Podemos pensar en la arquitectura del programa como un diseño en 3 capas: la primera capa maneja la interacción con el usuario (menú e ingreso de datos); la segunda capa se encarga de orquestar la computación (conversión de grados a radianes, mandar a llamar otros procedimientos), y la tercera capa se encarga de proveer los métodos básicos para el cálculo de seno, coseno y cuadrados de un número de punto flotante.

La figura 5 solamente describe qué procedimientos mandan a llamar otros procedimientos, no describe cómo se comparten información entre sí. Veamos ahora la estrategia que seguimos para este problema.

4.3. Estrategia de paso de parámetros

A diferencia de los lenguajes de alto nivel, con ensamblador no contamos con una manera concreta para pasar parámetros a otros procedimientos; tenemos la libertad de escoger nuestra propia estrategia. Nosotros intentamos ser consistentes con la convención que decidimos usar para pasar parámetros y regresar resultados. Usamos dos mecanismos, dependiendo del tipo de dato que deseamos manejar: valores enteros y valores de punto flotante.

4.3.1. Valores de punto flotante

Los valores de punto flotante se pasan y regresan a través del stack del FPU. El procedimiento que hace la llamada carga los valores en ST(0) —en ocasiones, también en ST(1)— antes de llamar al procedimiento, y el procedimiento que es llamado deja sus resultados en el stack también, antes de regresar. Cada procedimiento tiene, en forma de comentarios, la descripción del estado de stack que espera antes de iniciar el procedimiento y de cómo va a quedar el stack al final.

Por ejemplo, el procedimiento `EvalSinCos` espera dos copias del valor de x, una en ST(0) para calcular $\sin(x)$ y otra en ST(1) para calcular $\cos(x)$, y regresa tres valores: $\sin^2(x) + \cos^2(x)$

en ST(0), $\cos(x)$ en ST(1) y $\sin(x)$ en ST(2). Lo anterior está documentado a manera de comentario en `eval-sin-cos.asm`:

```
; Entrada FPU:   ST(0) = x_sin, ST(1) = x_cos
; Salida  FPU:   ST(0) = f1(x),  ST(1) = cos(x),  ST(2) = sin(x)
```

4.3.2. Valores enteros

Los argumentos enteros se pasan a través del *runtime stack*. El procedimiento que hace la llamada hace un `PUSH` del valor que quiere pasar al procedimiento que está llamando y luego ejecuta la instrucción `CALL`. El procedimiento llamado accede a los parámetros apoyándose en `ESP` y tomando en cuenta la dirección de retorno y cualquier otro registro que se haya guardado antes de acceder al parámetro.

Por ejemplo, en `SinFloat`, después de almacenar EAX, ECX y EDX (lo que toma 12 bytes), podemos encontrar el parámetro n —el número de iteraciones a realizar— en `[ESP + 16]`, pues se suman los 12 bytes de los registros almacenados más los 4 bytes de la dirección de retorno.

Al momento de finalizar un procedimiento, hay dos estrategias que usamos en el código. La primera estrategia, usada en procedimientos como `SinFloat`, `CosFloat`, `EvalGrados` e `ImprimeGrados`, consiste en limpiar los propios argumentos del *stack* al usar `RET 4`. La segunda estrategia, que usamos en `ConvierteRangoSin` y `ConvierteRangoCos`, consiste en regresar el resultado en la ubicación de memoria en la que estaba el parámetro pasado al procedimiento (es decir, `[ESP + 8]` después de haber guardado EAX). Así, el procedimiento que hizo la llamada puede acceder al resultado al hacer `POP`. Estos procedimientos simplemente usan `RET` sin un operando.

5. Algunos detalles de la implementación

En esta sección, veremos algunos detalles de cómo implementamos el código. Por razones de espacio, no pondremos todo el código fuente aquí, pero se puede acceder al código completo en GitHub (Márquez Luján, 2026).

5.1. Recurrencia en las series de Taylor

El núcleo computacional del programa es el par de procedimientos que calculan las series de Taylor de seno y coseno: `SinFloat` y `CosFloat`. Ambos procedimientos siguen el mismo patrón, pero difieren solamente en el término inicial y en el cálculo del denominador.

En lugar de calcular cada término de manera independiente —lo cual involucraría calcular factoriales y potencias demasiado grandes—, nuestra implementación usa una relación de recurrencia que sirve para calcular un término basándose en el anterior. Es decir, si x_k es el k-ésimo término de la serie de Taylor de $\sin(x)$, en lugar de usar (7) para calcular el siguiente término, usamos la siguiente relación:

$$x_{k+1} = x_k \frac{-x^2}{(2k+2)(2k+3)}, \tag{12}$$

partiendo de $x_0 = x$. De manera similar, en lugar de usar (9) para $\cos(x)$, usamos

$$x_{k+1} = x_k \frac{-x^2}{(2k+1)(2k+2)}, \tag{13}$$

partiendo de $x_0 = 1$. Como mencionamos anteriormente, la principal diferencia es el término inicial y el cálculo del denominador.

La recurrencia es mucho más eficiente y estable numéricamente. Al usarla, nos evitamos calcular valores intermedios como $13! = 6,227,020,800$ para $k = 6$, y en su lugar vamos multiplicando cada término por algo que va reduciendo su magnitud.

El siguiente extracto del código de `SinFloat` ilustra cómo se implementa la recurrencia:

```
; ST(0)=term, ST(1)=sum, ST(2)=-x^2, ST(3)=x
FMUL ST(0), ST(2)       ; term *= -x^2

; term /= (2k+2)(2k+3)
PUSH EAX

FILD DWORD PTR [ESP]    ; ST(0)=denom, ST(1)=term, ST(2)=sum, ...
ADD  ESP, 4
FDIVP ST(1), ST(0)      ; term = term / denom, pop denom
; Ahora ST(0)=new_term, ST(1)=sum, ST(2)=-x^2, ST(3)=x

FADD ST(1), ST(0)       ; sum += term
```

Un elemento importante aquí es el uso de `FILD DWORD PTR [ESP]` para convertir al denominador, que es un entero que calculamos usando shifts y multiplicaciones, a un valor de punto flotante en el FPU *stack*. El denominador, almacenado en EAX, es cargado brevemente al *runtime stack* solo para que la instrucción `FILD` pueda leerlo en memoria, y luego `ESP` se ajusta inmediatamente después. Esto evita la necesidad de una variable de memoria para el denominador.

El FPU *stack* es manejado con cuidado en el ciclo del procedimiento. Mantenemos cuatro valores simultáneamente: el término actual, la suma de términos al momento, la constante $-x^2$ y el valor original x (para $\sin(x)$). Los comentarios a lo largo del código fueron claves para no perdernos. El ciclo termina después de un número fijo de iteraciones, que se determina por el parámetro n pasado al procedimiento, y al final del procedimiento se descartan todos los valores intermedios; solamente dejamos el resultado de la suma en ST(0).

5.2. Conversión de grados a radianes y reducción de rango

El procedimiento `EvalGrados` se encarga de orquestar tres cosas: la conversión de grados a radianes, reducción o simplificación del rango, y finalmente la evaluación de las funciones trigonométricas. Este procedimiento convierte grados a radianes siguiendo una estrategia de tratar con los datos enteros primero; es decir, si tenemos que la conversión de grados a radianes está dada por

$$\text{radianes} = \frac{\text{grados} \cdot \pi}{180},$$

el código primero divide los grados entre 180 y luego multiplica el resultado por π.

Implementamos la conversión de la siguiente manera:

```
PUSH EAX
FILD DWORD PTR [ESP]          ; ST(0) = deg (como float)
POP  EAX
FIDIV val180                  ; ST(0) = deg / 180
FLDPI                         ; ST(0) = pi,  ST(1) = deg/180
FMULP ST(1), ST(0)            ; ST(0) = rad_true
```

Como podemos ver, cargamos los grados en el FPU *stack* de una manera similar a lo que hicimos en la sección anterior; pero el valor de 180 lo tenemos en la sección de `.DATA` como `val180 DWORD 180`.

`EvalGrados` tiene que hacer dos conversiones de grados a radianes: una antes de simplificar el rango y otra después.

La reducción de rango está a cargo de `ConvierteRangoSin` y `ConvierteRangoCos`. Para $\sin(x)$, nos apoyamos de la siguiente tabla para hacer la reducción:

Rango de entrada	Valor regresado	Identidad usada
$0^\circ \leq x \leq 90^\circ$	x	N/A
$90^\circ < x \leq 180^\circ$	$180^\circ - x$	$\sin(x) = \sin(180^\circ - x)$
$180^\circ < x \leq 270^\circ$	$180^\circ - x$	$\sin(x) = -\sin(x - 180^\circ) = \sin(180^\circ - x)$
$270^\circ < x \leq 360^\circ$	$x - 360^\circ$	$\sin(x) = \sin(x - 360^\circ)$

Figura 6: Reducción de rango para $\sin(x)$.

Para el coseno, la simplificación se apoya en el hecho de que $\cos(x) = \cos(360^\circ - x)$, por lo que el rango de salida es de $[0^\circ, 180^\circ]$; si $x \leq 180^\circ$, no se altera; si $x > 180^\circ$, entonces regresamos $360^\circ - x$.

5.3. Retos de implementación y consideraciones

Al programar este proyecto, nos enfrentamos a varios retos. El desarrollo de un proyecto en lenguaje ensamblador es considerablemente más complicado que cuando se usa un lenguaje de alto nivel. En particular, encontrar *bugs* en el código es más difícil porque no contamos con las herramientas que usamos normalmente, como *debuggers*.

Un error que fue especialmente difícil detectar fue una confusión en el momento de usar `FDIVP ST(1), ST(0)`. Originalmente, habíamos escrito los operandos al revés (es decir, `FDIVP ST(0), ST(1)`), lo cual realmente no tiene sentido, pues la idea de agregar el *pop* al final de la operación es que se libere ST(0), por lo que no tiene sentido almacenar el resultado de la división ahí y luego liberarlo en la misma instrucción. Esta operación está en el núcleo de los bucles para calcular el seno y coseno, por lo que tuvimos que ir siguiendo cuidadosamente el flujo de las operaciones, agregando la impresión del stack del FPU a consola en prácticamente todos los pasos para poder encontrar el error.

En general, los principales retos vinieron del manejo del stack del FPU. Incluso para ser lenguaje ensamblador, es muy diferente a trabajar con valores enteros. Además, la naturaleza circular del stack del FPU nos obliga a tener mucho cuidado al querer cargar un valor; el espacio es muy limitado y hay que cuidarnos de no sobrescribir valores. En nuestro código, intentamos no manejar más de 5 valores en el FPU stack, lo cual ya está peligrosamente cerca de los 8 valores permitidos.

A la hora de interrumpir el ciclo que calcula los términos de la serie de Taylor, nos enfrentamos a dos alternativas:

1. Al llegar a un número determinado de iteraciones, o
2. Cuando el k-ésimo término sea menor que una tolerancia determinada.

Nosotros nos inclinamos por la primera opción. Como vimos con la ecuación (10), $k = 10$ iteraciones son suficientes para valores adecuados de x. Además, con esta elección nos ahorramos el problema de comparar valores de punto flotante en cada iteración, y, en caso de un error, nos aseguramos de no estar en el ciclo eternamente. La reducción de rango que discutimos en la sección 5.2 nos permite asegurarnos de que x es un valor adecuado, por lo que no perdemos precisión al fijar el número de iteraciones.

La reducción de rango trae consigo un aumento en el número de operaciones que hay que realizar. Hay varias instrucciones en los procedimientos que usamos para esta conversión. Sin embargo, consideramos que vale la pena hacer este cambio, pues esto permite que la serie converja en menos términos, lo que en última instancia termina reduciendo el número de instrucciones necesarias.

Otra consideración es que la identidad $f_1(x) = \sin^2(x) + \cos^2(x)$ debería ser exactamente 1. En la práctica, $f_1(x)$ no siempre es exactamente 1, pero no por eso deja de servir como diagnóstico para saber cuando algo está mal. Los errores de aproximación por serie de Taylor, además de los errores por tener precisión finita, hacen que la operación rara vez el valor sea 1 con total precisión, pero, como veremos en las capturas de pantalla, si mostramos solamente los primeros decimales, normalmente sí está lo suficientemente cerca en nuestro programa.

6. Consideraciones de Linux

Irvine (2015) se concentra en sistemas de Windows y MASM. Sin embargo, como reto adicional, nosotros decidimos intentar ejecutar todo en un sistema basado en Linux. Realizamos todo el desarrollo de este proyecto en una máquina con un procesador Intel i5-4570R y con sistema operativo Ubuntu 22.04.5 LTS x86_64. La figura 7 muestra la salida del comando `neofetch` para describir con detalle el sistema utilizado.

El ensamblador que utilizamos fue UASM (Habus & Hankinson, 2022), que es compatible con MASM y está basado en JWasm. Una vez creados los objetos, para enlazarlos nos apoyamos del compilador de C, `gcc`. Al momento de enlazar, dependemos fuertemente de la librería estática `IrvingT` (Chung, 2016). Esta librería es una reimplementación de la librería `Irvine32` de Irvine (2015).

Para este proyecto en específico, tuvimos que modificar un poco el código de Chung (2016) para incluir las siguientes funciones que ofrece `Irvine32`:

- `WriteFloat`
- `ReadFloat`
- `ReadInt`
- `GetMseconds`

En realidad, las modificaciones necesarias consisten en agregar las llamadas correctas en el código fuente de C y agregar el procedimiento al archivo `.S`. Por ejemplo, para agregar el

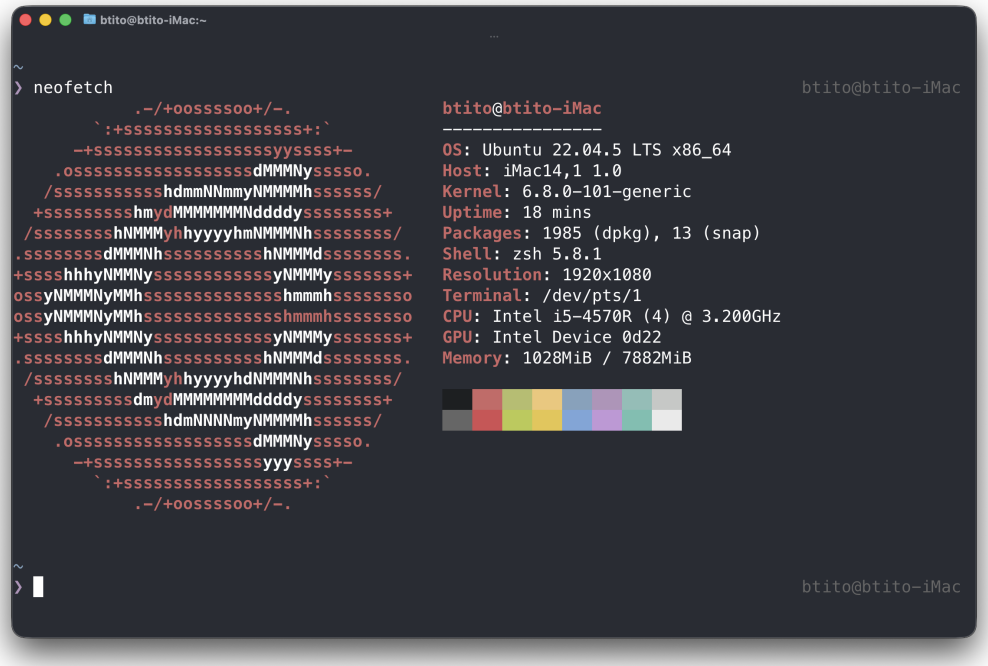


Figura 7: Salida del comando `neofetch`.

procedimiento `WriteFloat`, hay que agregar lo siguiente a `irvingt-real.c`:

```
#ifndef IRVINGT_HAVE_ALONG
void WriteFloat_Real(double x) {
  printf("%.10f", x);
  fflush(stdout);
}
#endif
```

y las siguientes instrucciones a `irvingt.S`:

```
#ifndef IRVINGT_HAVE_ALONG
.globl WriteFloat
.type WriteFloat, @function

WriteFloat:
  pushal
  pushfl

  subl $8, %esp
  fstl (%esp)
  call WriteFloat_Real
  addl $8, %esp

  popfl
  popal

  ret
#endif
```

Así, ya podemos hacer llamadas a `WriteFloat` desde nuestro código.

Para facilitar el ensamblado y enlazado del código, y no tener que estar ejecutando la instrucción una vez por cada módulo, creamos un Makefile que automatiza todo el proceso.

El contenido es el siguiente:

```
# Tools
ASM = uasm
CC  = gcc

# Flags
ASMFLAGS = -I./Irvine -zcw -elf

# Config
TARGET  = asm-trig
MODULES = main sine-n cosine-n square-float eval-sin-cos pide-radianes \
    convierte-rango eval-grados pide-grados imprime-grados tabla-grados

OBJS = $(addsuffix .o, $(MODULES))

# Rules
all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) -m32 $(OBJS) -o $(TARGET) -L./irvingt -l:irvingt.a

%.o: %.asm
    $(ASM) $(ASMFLAGS) -Fo=$@ $<

clean:
    rm -f *.o $(TARGET)
```

Esencialmente, estamos ejecutando `uasm` con las banderas correspondientes una vez por cada módulo `.asm` que tenemos para crear el correspondiente `.o`; y al final enlazamos todo con `gcc`. La creación de un Makefile también ayuda a que solamente se ejecute `uasm` para los archivos fuente que hemos modificado, no todo el tiempo. Así, con solo ejecutar `make` en la terminal, se crea automáticamente un ejecutable.

7. Capturas de pantalla

Las capturas de pantalla aquí mostradas son de una ejecución del programa en un ambiente de Windows, aunque, inicialmente, el programa fue diseñado para ejecutarse en un sistema Linux. Aunque vimos los detalles del desarrollo y la implementación particulares a Linux en la sección 6, ahora mostraremos lo que ve el usuario final del proyecto.

Para ejecutar en Windows, fue necesaria una pequeña modificación a la hora de declarar los procedimientos externos: tuvimos que agregar un `@0` al final del nombre del procedimiento. Pero con eso, Visual Studio fue capaz de ensamblar y enlazar todo. El código funciona exactamente igual que en Linux, con la única consideración de que no muestra bien las tildes.

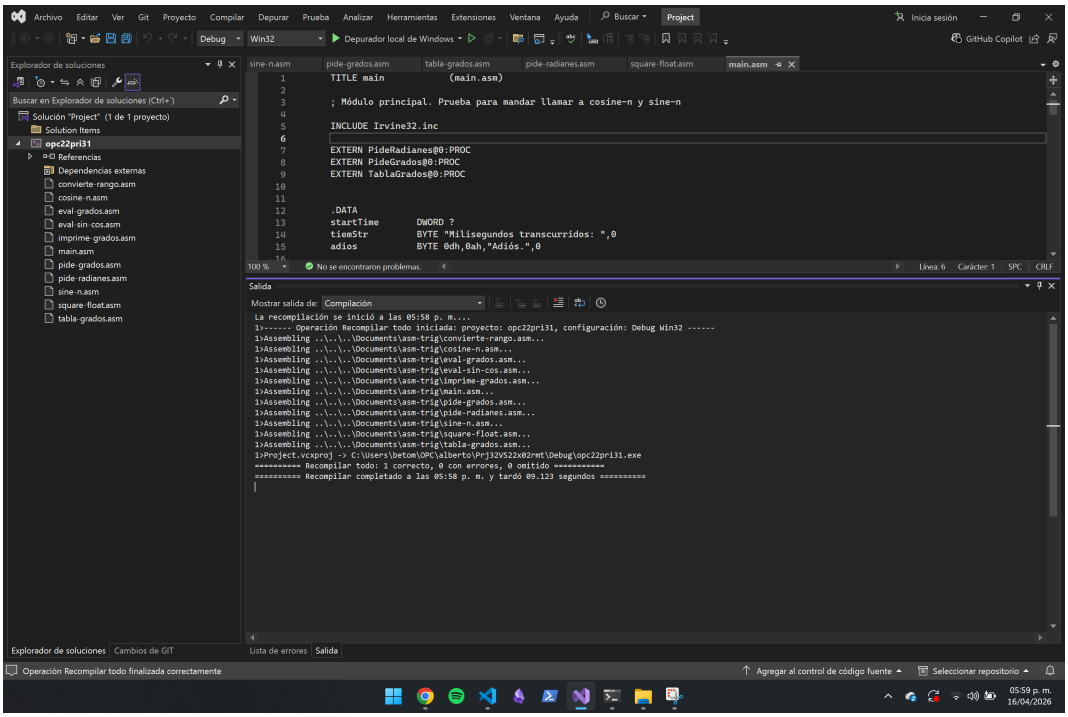


Figura 8: Compilación exitosa del proyecto en Visual Studio.

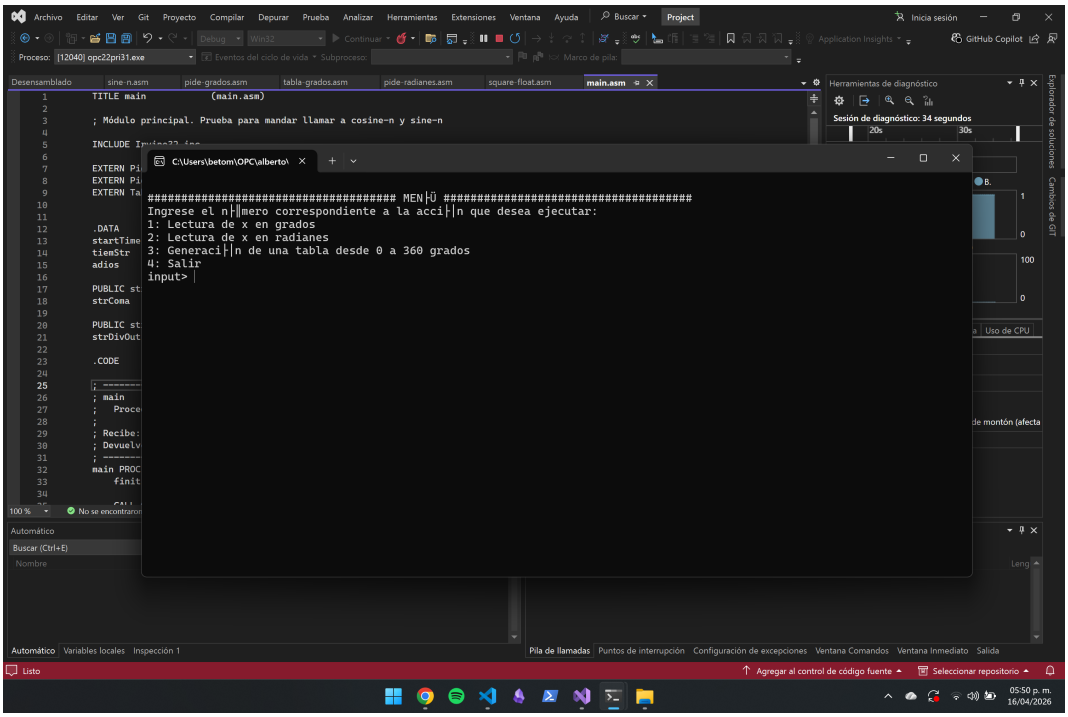


Figura 9: Vista del menú principal.

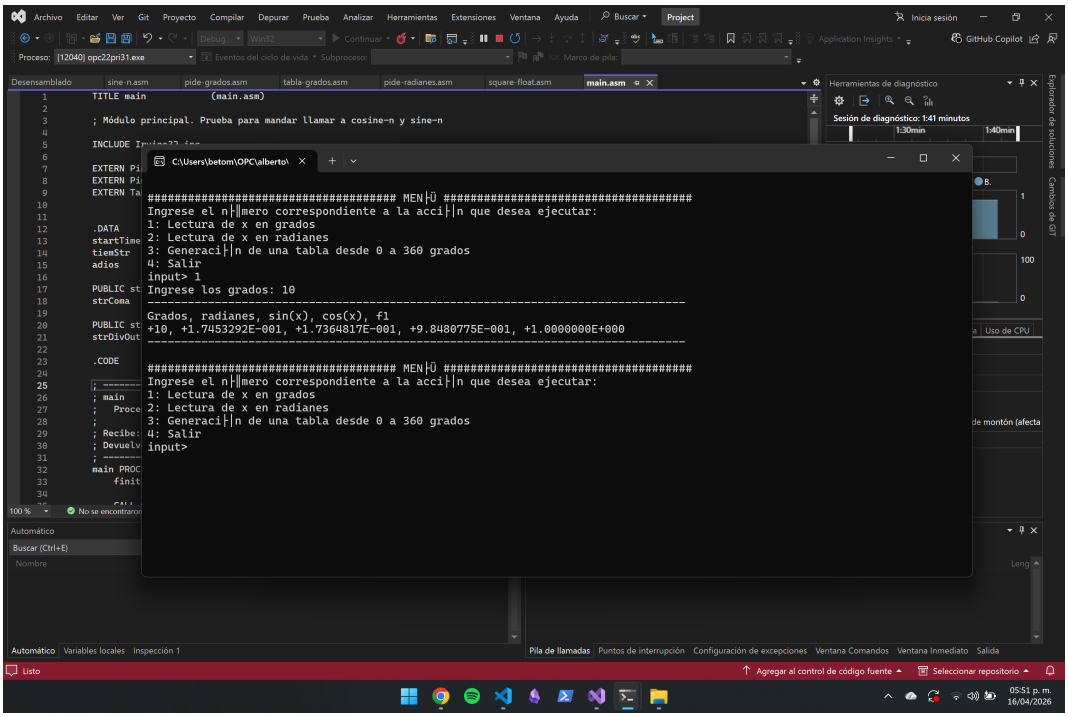


Figura 10: Ejecución de la opción 1: lectura de x en grados y evaluación.

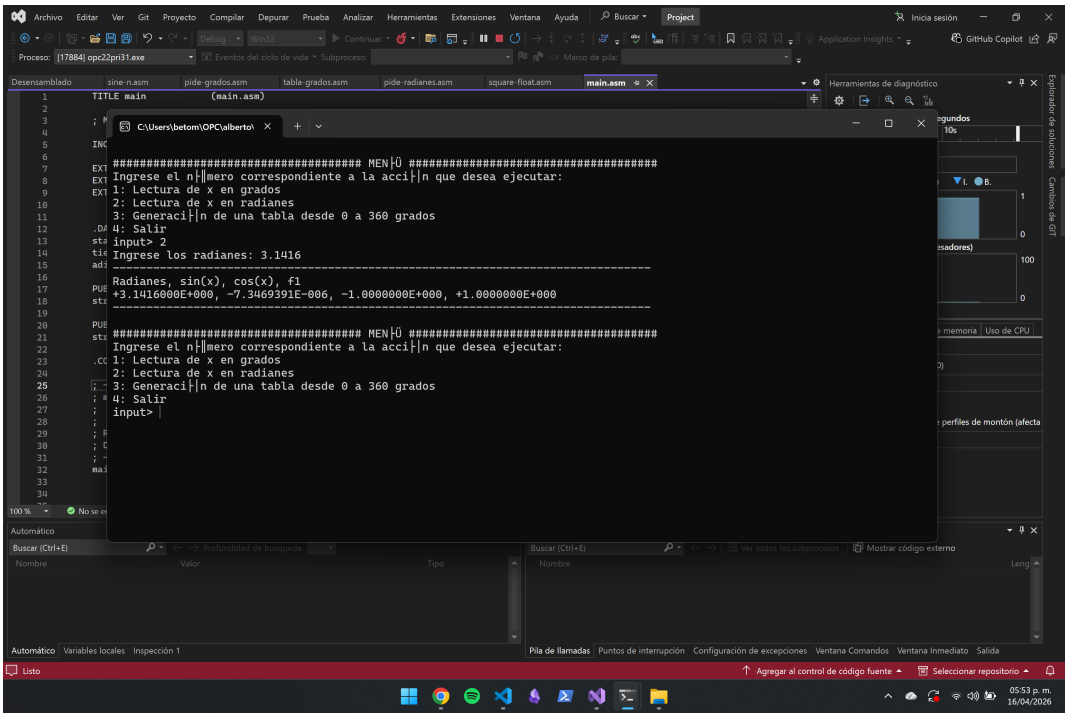


Figura 11: Ejecución de la opción 2: lectura de x en radianes y evaluación.

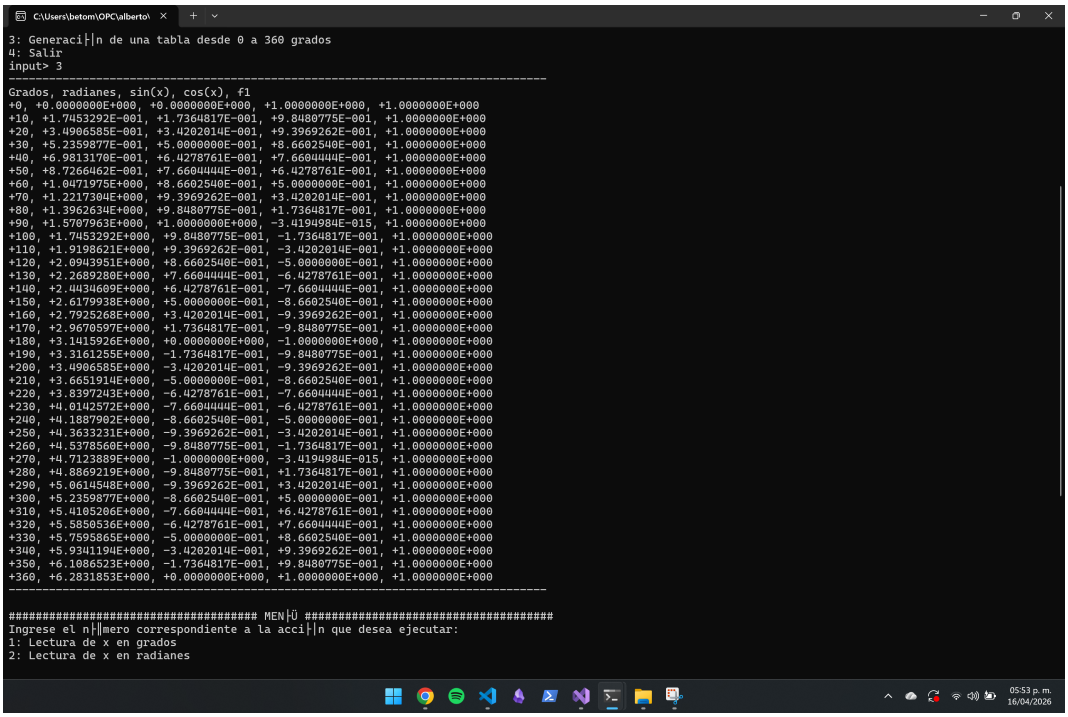


Figura 12: Ejecución de la opción 3: tabla de 0° a 360°.

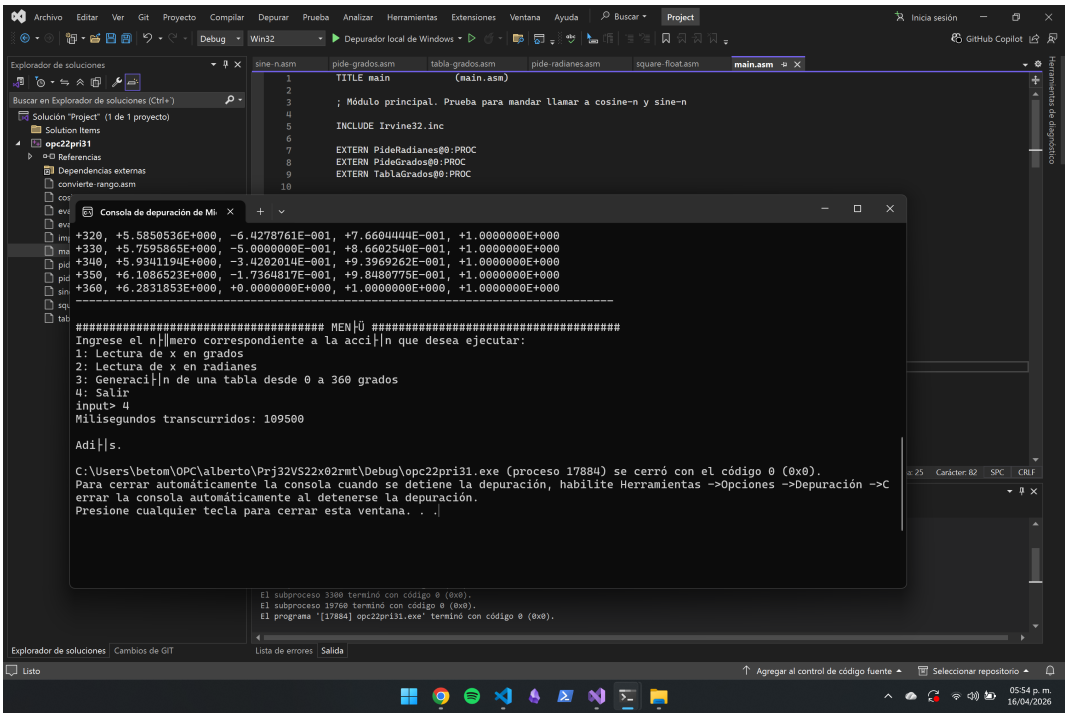


Figura 13: Ejecución de la opción 4: salir del programa.

Referencias

Chung, P. (2016). IrvingT. https://github.com/ObjectBoxPC/irvingt

Habus, B., & Hankinson, J. (2022). UASM - Macro Assembler. https://www.terraspace.co.uk/uasm.html

Irvine, K. R. (2015). *Assembly Language for x86 Processors* (7.ª ed.). Pearson.

Márquez Luján, J. A. (2026). Trigonometry functions in ASM. https://github.com/betomqz/asm-trig

Ríos, R. (2023). Notas para la clase de Org. y Prog. de Computadoras.

Thomas, G. B. (2010). *Cálculo una variable* (12.ª ed.). Pearson.